CLAUDE CERTIFIED ARCHITECT — FOUNDATIONS

Objective 30: Reducing false positives in automated review

> **Exam objective:** Reduce automated review false positive rates by supplying project-specific conventions, accepted patterns, and exclusion criteria as persistent context applied on every review.

Q1.

An automated code review agent repeatedly flags the team's custom logging wrapper as "dead code" because it does not recognize the framework convention. What is the best remediation?

A. Disable all dead-code detection permanently B. Provide persistent project-specific context explaining that the logging wrapper is an accepted pattern used via reflection or framework hooks C. Increase temperature so the model is less strict D. Ask reviewers to ignore the warnings manually

Answer: B **Explanation:** False positives often arise when the model lacks local conventions. Supplying persistent context about accepted patterns helps the reviewer distinguish intentional design from genuine defects without turning off useful checks entirely.

Q2.

An automated review agent flags generated API clients on every pull request for naming-style violations. What is the best way to reduce this noise?

A. Tell the model to be nicer B. Add persistent exclusion criteria indicating that generated code paths should not be reviewed for style findings C. Rename the generated files manually after each build D. Accept the noise because generated files are still code

Answer: B **Explanation:** Generated artifacts are a classic exclusion candidate. Persistent review context should specify which directories or file classes are exempt so the model applies that rule consistently across reviews.

Q3.

Your team intentionally uses dynamic SQL builders with parameterization, but the review model keeps flagging any SQL string construction as injection risk. What is the best context to add?

A. A project-specific note describing the safe query-builder pattern, how parameter binding is enforced, and which constructs are acceptable B. A blanket rule to never mention security issues C. A request for more verbose explanations D. A statement that SQL injection no longer exists

Answer: A **Explanation:** The fix is not to suppress security review globally, but to teach the model the local safe pattern and its boundaries. This reduces false positives while still allowing real unsafe cases to be flagged.

Q4.

Why should accepted project conventions be provided as persistent context rather than repeated ad hoc in individual prompts?

A. Persistent context improves consistency so the model applies the same exceptions and norms on every review B. Persistent context makes the model ignore all future changes automatically C. Repeating context each time is impossible with LLM APIs D. Persistent context eliminates the need for human review entirely

Answer: A **Explanation:** False-positive reduction depends on consistent application of local knowledge. If conventions are only mentioned occasionally, reviews become uneven and noisy.

Q5.

A mobile team intentionally stores feature flags in a central singleton during app startup. The reviewer keeps flagging this as global mutable state. What is the best improvement?

A. Add project context describing the approved startup initialization pattern and when singleton usage is allowed B. Remove all singleton warnings everywhere C. Rewrite the application architecture solely to satisfy the review model D. Increase max_tokens so the model reads more code

Answer: A **Explanation:** If the pattern is an accepted architectural exception, encode that policy explicitly. Blanket suppression removes useful signal, while changing architecture to fit the tool is not the right first response.

Q6.

A review system often flags files under `migrations/` for missing tests, even though the team's policy exempts one-time database migrations. What persistent rule best addresses this?

A. "Ignore all database code" B. "Do not raise missing-test findings for files under `migrations/` because they are deployment artifacts covered by separate validation procedures" C. "Only review files in alphabetical order" D. "Assume migrations are always safe"

Answer: B **Explanation:** Good exclusion criteria are specific and bounded. The rule explains both scope and rationale, reducing repeated noise without exempting unrelated database logic.

Q7.

An automated review flags HTTP 404 handling as an error path, but in your API gateway 404 is an expected negative lookup result. How should you reduce false positives?

A. Add persistent context documenting that 404 is an accepted business outcome for certain endpoints and should not be flagged unless mishandled B. Tell the model to never mention HTTP status codes C. Change all endpoints to return 200 D. Convert every 404 to 500 for clarity

Answer: A **Explanation:** Project-specific semantics matter. The review agent should understand when a status code is expected behavior versus a bug, and persistent context is the right place to encode that distinction.

Q8.

A team uses snapshot tests intentionally for UI regression coverage, but the model keeps criticizing them as brittle. What is the best approach?

A. Add project guidance that snapshot tests are an accepted pattern in specific UI packages, along with criteria for when warnings are still appropriate B. Disable all test-related review checks C. Replace snapshot tests immediately D. Ask the model to focus only on backend code

Answer: A **Explanation:** The goal is calibrated review, not silence. By describing where snapshot tests are sanctioned and their limits, you reduce false positives while retaining useful scrutiny in inappropriate contexts.

Q9.

Which type of information is most valuable for lowering false positives in automated review?

A. Generic advice like "be careful" B. Specific conventions, approved idioms, known safe abstractions, and explicit exclusions tied to the project C. A list of every programming language keyword D. Longer chain-of-thought requests

Answer: B **Explanation:** False positives are usually caused by missing local context, not lack of generic caution. The most effective inputs are concrete, project-specific rules that help the model distinguish normal patterns from real problems.

Q10.

A review bot flags every usage of a homegrown retry helper as "possible infinite loop," even though the helper enforces capped retries and backoff. What should persistent context include?

A. A note that the retry helper is approved, capped, and should only be flagged when callers bypass its standard configuration B. An instruction to ignore all loops C. A request for more concise comments D. A rule to approve all networking code automatically

Answer: A **Explanation:** This targets the precise source of the false positive while preserving the model's ability to catch genuinely unsafe retry logic elsewhere. Overbroad suppression would hide real issues.

Q11.

Your organization stores test fixtures containing fake secrets under `testdata/`, and the review model keeps raising credential-exposure findings. What is the best tuning?

A. Exclude `testdata/` from all review forever without any conditions B. Provide persistent context that fake secrets in designated fixture paths are allowed test data, but real-looking credentials outside those paths should still be flagged C. Turn off secret detection globally D. Rename the folder randomly each sprint

Answer: B **Explanation:** The context should carve out a narrowly defined safe zone rather than disabling an entire class of findings. This reduces fixture-related noise while protecting real source paths.

Q12.

A monorepo contains legacy modules that intentionally violate current style rules pending migration. The review bot floods PRs touching those modules. What is the best context strategy?

A. Define scoped exclusions or reduced-severity rules for identified legacy paths until migration is complete B. Ignore all style rules across the repository C. Merge the PRs without any review D. Delete the legacy modules immediately

Answer: A **Explanation:** Scoped, path-based policy lets the model adapt to transitional realities without losing standards elsewhere. Repository-wide suppression would create unnecessary blind spots.

Q13.

Why do exclusion criteria need to be explicit rather than implied from prior reviews?

A. Models do not reliably infer stable project policy from scattered historical examples alone B. Implied rules are always stored permanently by default C. Explicit exclusions make the model unable to review new code D. Prior reviews cannot contain text

Answer: A **Explanation:** Without explicit persistent context, the model may treat prior outcomes as isolated examples rather than binding policy. Clear rules improve consistency and reduce recurring false positives.

Q14.

A security review assistant keeps flagging internal service-to-service calls for missing end-user authentication, but the project uses mutual TLS plus service identity instead. What is the best persistent context addition?

A. Document the trusted internal auth model, including that mTLS and service identity satisfy authentication requirements for specific internal call paths B. Tell the model to stop reviewing authentication entirely C. Replace mTLS with anonymous access D. Rename the services to indicate trust

Answer: A **Explanation:** The model needs the project's actual security architecture to judge findings correctly. Documenting the approved internal trust pattern reduces false alarms while preserving meaningful security review.

Q15.

A review platform lets you attach a standing instruction set to every repository scan. Which instruction most directly serves the objective of reducing false positives?

A. "Find as many issues as possible, regardless of team norms" B. "Apply these repository-specific conventions, approved patterns, and exclusion rules on every review before deciding whether

behavior is problematic" C. "Focus only on comments and commit messages" D. "Use the longest possible response format"

Answer: B **Explanation:** The objective is specifically about persistent project-specific context applied on every review. That context helps the model calibrate findings to the repository's real conventions, reducing avoidable noise.